

Mono2iso

This app is to try to find 2-isohedral monohedral tilings.

Basically that means that the tiles will all be the same shape, but they will play different roles (called orbits) in the tiling. The 2-isohedral means there will be exactly 2 orbits in the tiling. Grunbaum and Shephard classified all isohedral tilings which have only one orbit. They did this by matching each tile's edges with the edge of the adjacent tile. There is a limited number of these.

For 2-isohedral tilings, since we have two types of tiles, we have to match edges between orbits including rotations and flips of tiles. We can also split up an edge into multiple edges. This gives us an infinite number of ways to tile.

How to use the app.

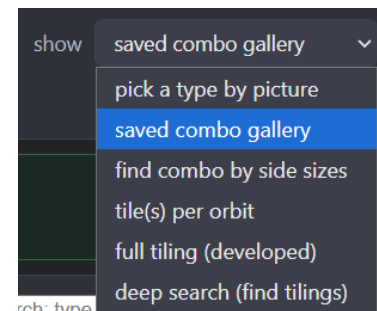
There are several ways to use the app.

- Look at example tilings in the gallery.
- Copy a given 2-isohedral, monohedral tiling. (Usually copying one of Joseph Myer's polyforms, But Escher did one example.)
- Make up tilings on your own:
 - Explore by computer
 - Explore manually.
- Given a tiling by any of the above methods, change the curves to look like an outline of something in the style of Escher. (This app doesn't support drawing inside of tiles, but can export to PNG or SVG.)

I'll expand these below.

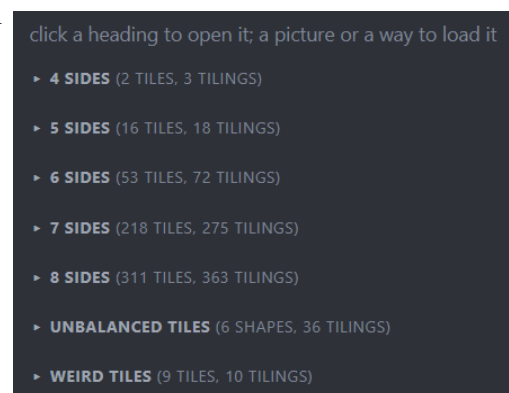
Look at example tilings in the gallery.

Choose the “saved combo gallery” mode from the dropdown menu.

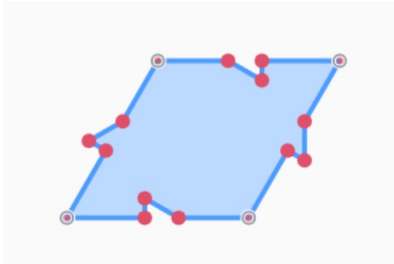


The gallery is sorted into sections. The first few are anisohedral tilings sorted by number of sides – from 4 to 8. We didn't find any 3 sided anisohedral tiles.

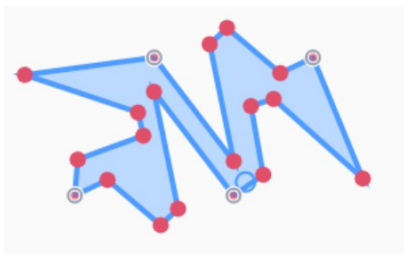
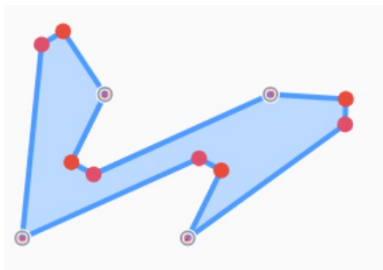
The next section is unbalanced tiles – which require that the ratio of tiles per orbit in large patches approach some ration other than 1:1. One of these hasn't been published before. The last section is weird tiles. These were ones that were a challenge to find a solution. Often we needed to use curved edges – since straight lines would make the boundary touch or overlap. Sometimes there was just a narrow range of edge lengths and angles that would work to close the tile.



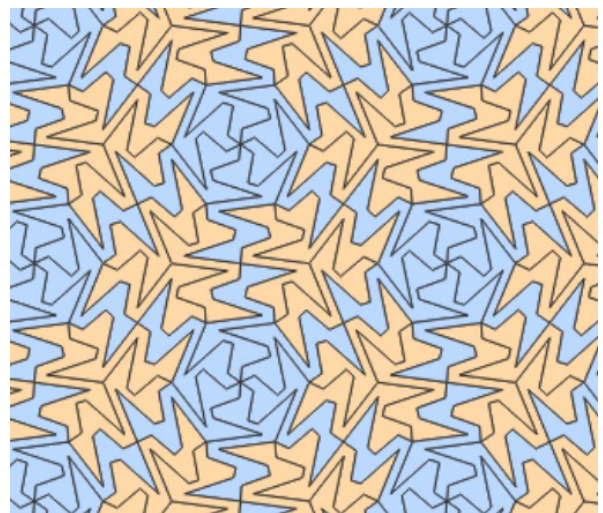
If you click a section, you will see examples. On the right is 4 sides expanded. Sometimes it is just a single tile, sometimes a tiling. Click “show all tiling ways” to further expand the single tiles. Click a tiling to open it. It defaults to “full tiling” mode when it opens. You can change the mode to “tiles per orbit” to see just a single tile. This is the easiest way to edit the tile shape. For example, below is the editable version of the rhombus. We can



drag the red dots to change the shape. If we drag on an edge away from a red dot, we will add a dot and move it. If we right click a dot, it deletes it. Here are some examples we can make with this tiling type.



If you click back to “full tiling” mode, you can see what it would look like as a tiling.



show all tiling ways

click a heading to open it; a picture or a way to load it

4 SIDES (2 TILES, 3 TILINGS)

#25577 #25593 - m4 rigid - 2 ways - same outline as 1 other tile

> #25577 sum=[1,0,0,0,0] refl off=1 · p4 · criterion 3

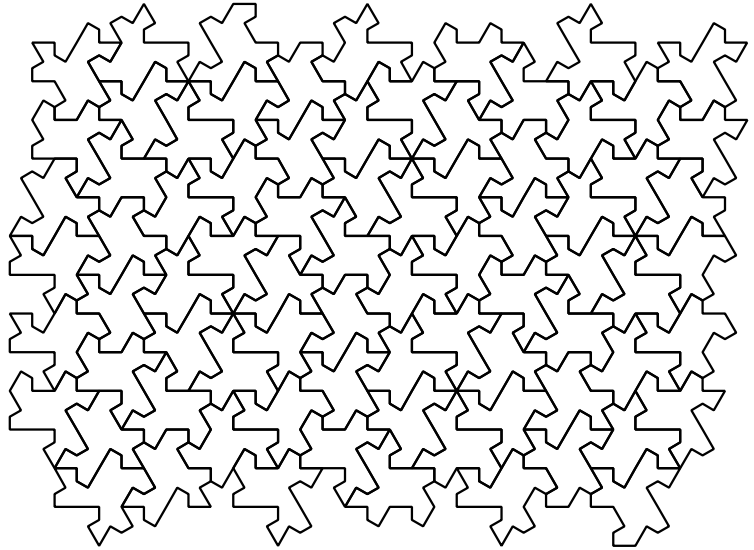
> #25593 sum=[1,0,0,0,0] direct off=0 · pg · criterion 9

Copy a given 2-isohedral, monohedral tiling.

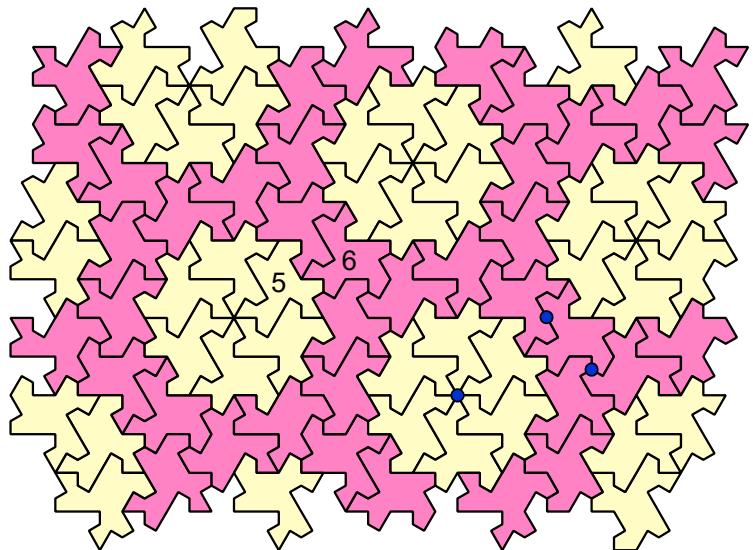
Say that you start with a 2-isohedral monohedral tiling. (Such as Joseph Myers has on his website.

<https://www.polyomino.org.uk/mathematics/polyform-tiling/>

The one at the right is a polykite with size $n = 14$ (and $k=2$, for 2-isohedral)

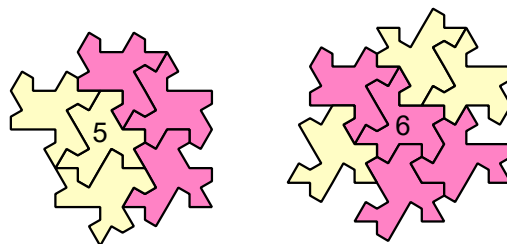


Then shade each orbit a different color. This helps to see the structure. I marked the rotation centers with blue dots. We have rotation centers of order 6, 3 and 2. There are no reflection lines of symmetry. The orbifold is “632.” We also count how many neighbor tiles each orbit has. I labeled a yellow tile “5” and a pink tile “6”.



orbifold = 632 (centers marked with blue dots)

Looking at the neighbors surrounding each tile, we look at how many of the neighbors are the opposite orbit. Each of these has three neighbors of the opposite orbit. We can use this information to find which tiling we are dealing with. This is a tiling in the $5_3 6_3$ family.



5_3

6_3

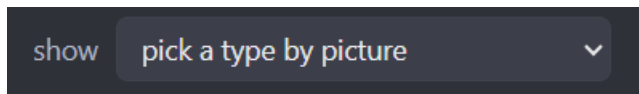
When the app opens you can see some boxes that show $5_2 6_2$.



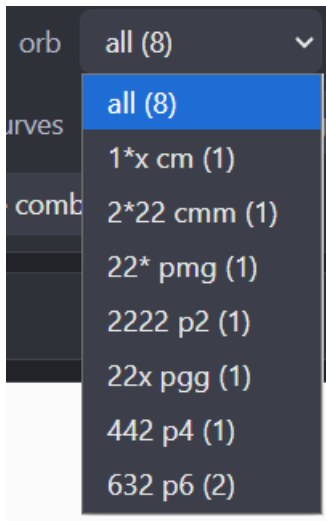
We need to change this to read $5_3 6_3$.



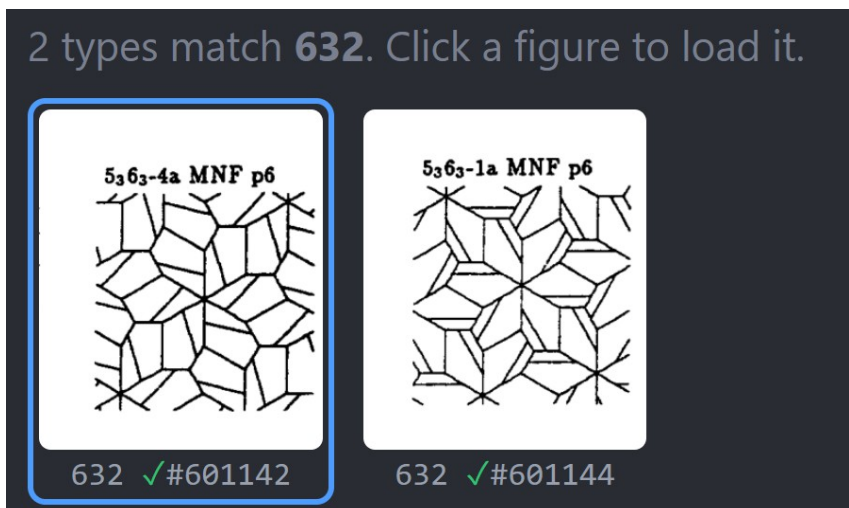
Set the mode to “Pick a type by picture.”



Here you would see 8 examples in the panel on the right. But we can narrow it further with the orbifold. Click the dropdown menu near “orb” and select 632.



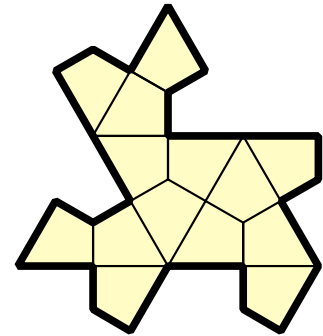
This leaves us with just two pictures to choose from in the panel. Now we have to study the pictures given and figure out which one matches our tiling. In this case, we either have a 5 sided figure at the 6 fold rotation center or a 6 sided figure. Our tiling has the 5 sided figure, so it is type #601142. (Note that the app has a short cut where you can enter this type number immediately, if you know it.) (also note the green checkmark, meaning it's possible to tile with the type. Red Xs mean impossible. Otherwise it's unknown.



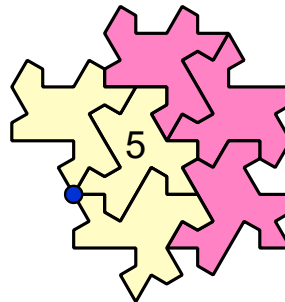
These shapes are polykites, so we can view them as composed of kite figures. Count the number of edges around the perimeter to get 24. (We count two edges, even if they are in a straight line.)

Set the size to 24.

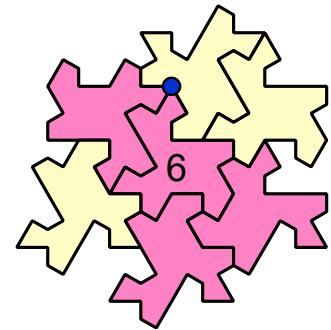
sides 24 ▾



Next, see how many edges are in each side. (I'm calling a edge of a kite an “edge” and the boundary between two tiles a “side”) So for the yellow orbit, starting at the blue dot and going around the tile clockwise, we have 8 edges in the first side, then 4 edges, followed by 2 and 2 and 8. Now find where that blue dot must be on the pink tile. For the pink orbit we have 4,2,4,4,2,8. Please note that both must add up to 24. If the orientation was opposite between orbits, we would go around the other direction for the pink orbit.



Clockwise
8,4,2,2,8



Clockwise
4,2,4,4,2,8

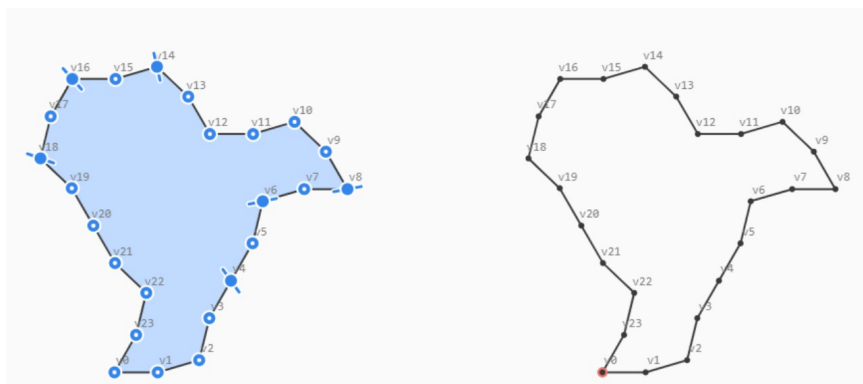
Set the mode to “find combo by side sizes.”

show find combo by side sizes ▾

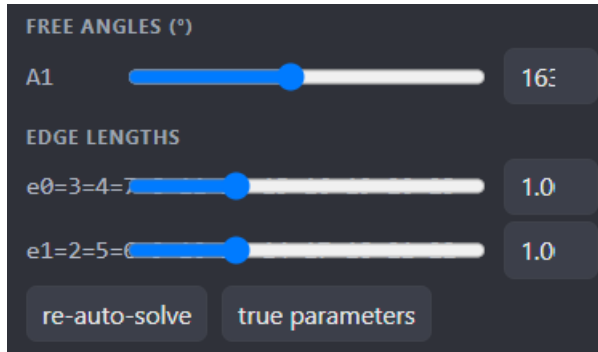
In the box for sides per orbit enter the lists, separated by a slash. Click on the anchor checkbox, since we started at matching vetices. Then click the “Show” button.

8,4,2,2,8 / 4,2,4,4,2,8 anchor ☒ show

The right panel shows options. Here we have only one option. Click it and you'll see a shape.

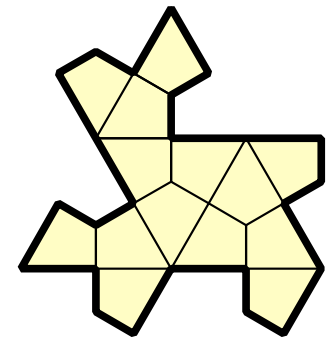


Note that the panel has an angle slider (A1) and a couple of length ratios.

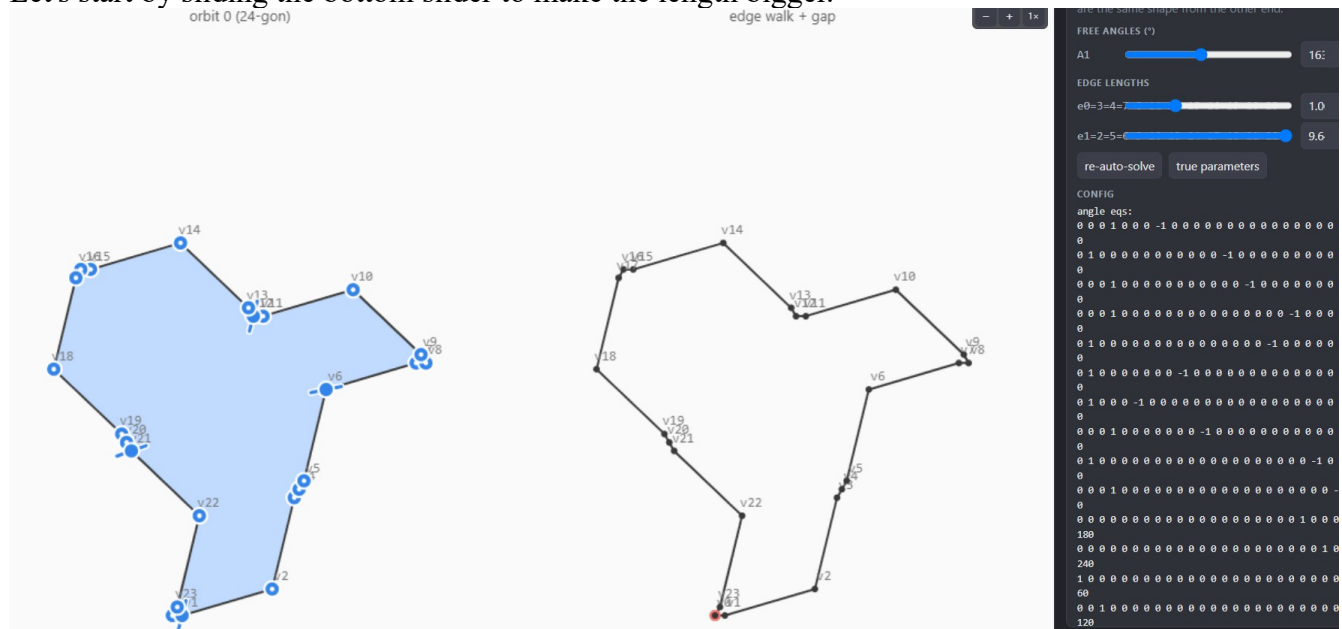


When we move these raw sliders, often the shape will break apart and not close. Then we have to adjust other sliders. There is a re-auto-solve button to make this easier.

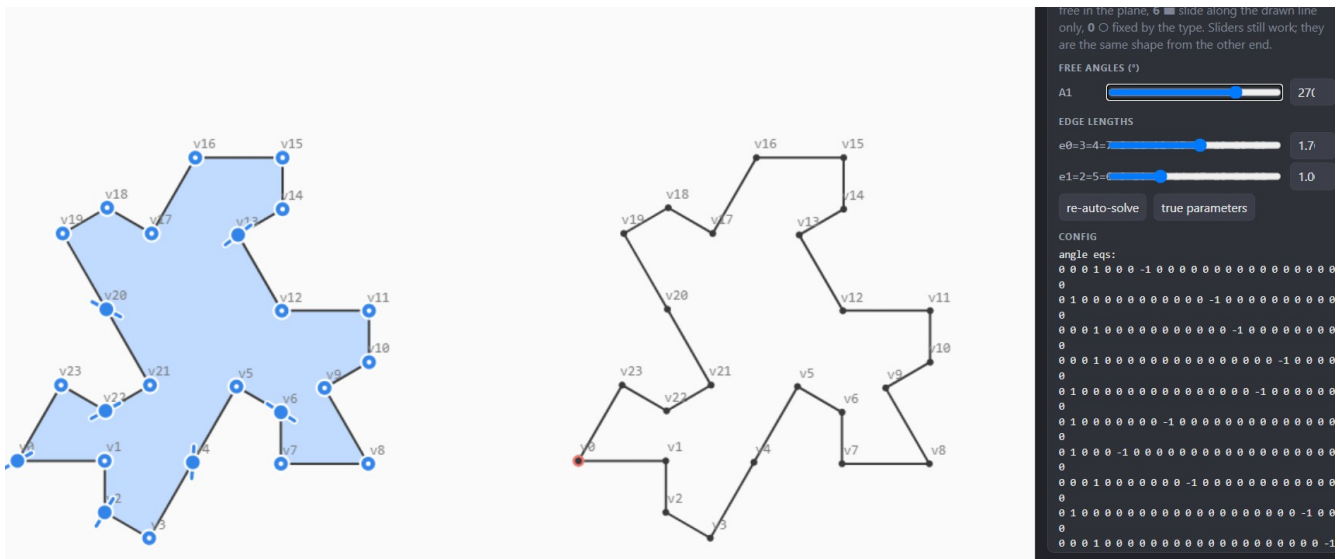
We are aiming at the figure at the left. Note how it has a couple of edges that are in a straight line. Also how some edges are a bit longer than others. In fact, since we are dealing with a 30-60-90 triangle, we know that the edge ratios are 1 to the square root of 3. So the ratio needs to be about 1:1.732 with two long edges lined up in two places.



Let's start by sliding the bottom slider to make the length bigger.

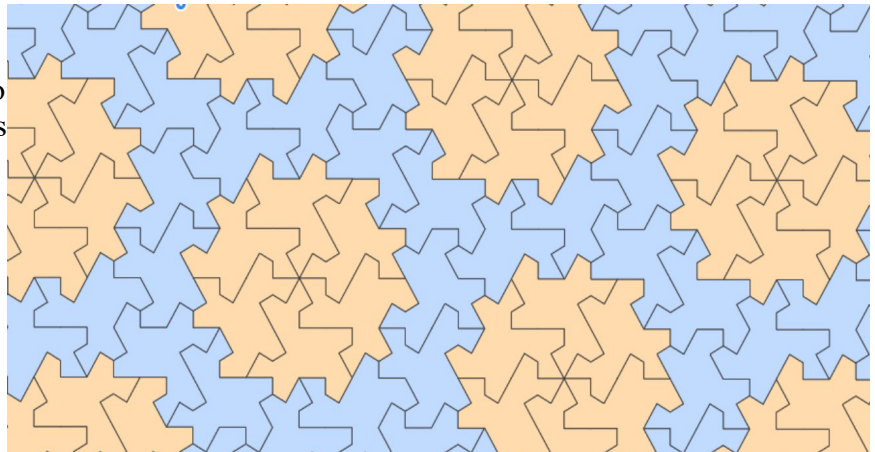


This is the wrong one to make longer. We can see now there are a couple of places where two short edges are in a straight line. We need two long edges in a straight line. So move the bottom slider back to 1 and move the middle slider to about 1.732

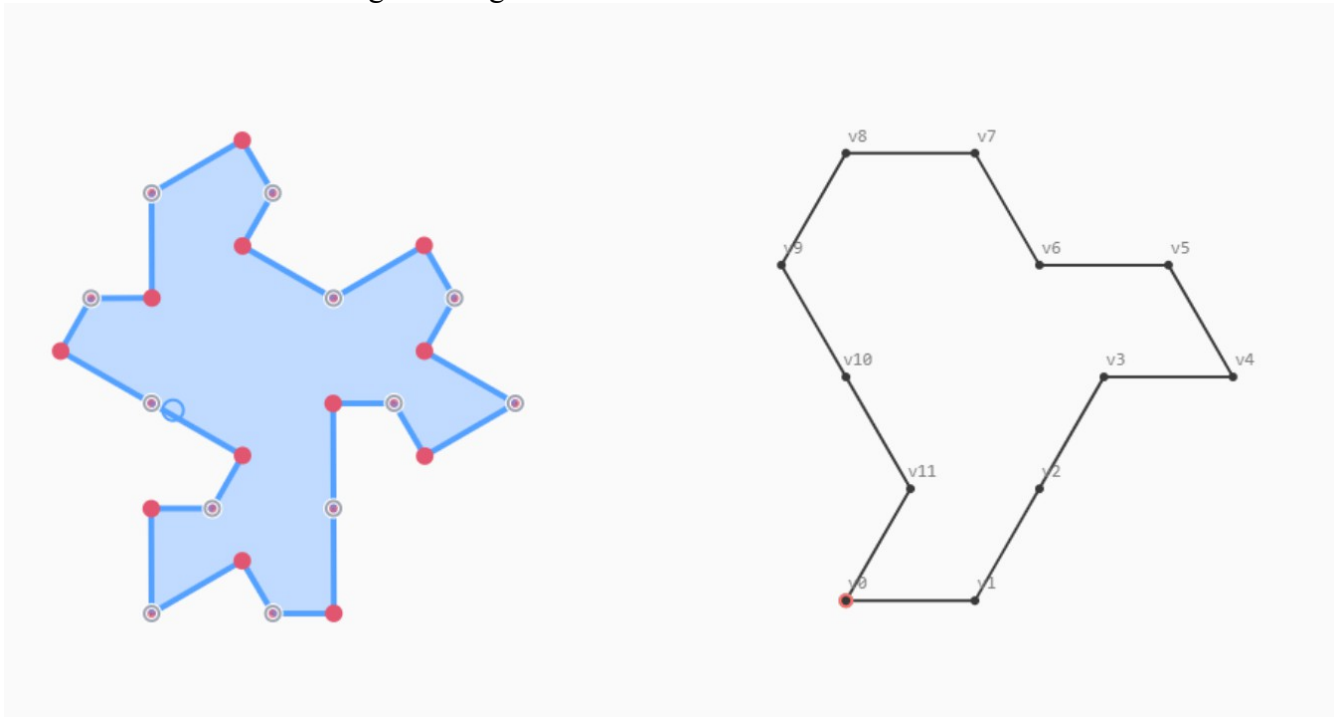


When we get to 270 degrees, this looks like what we are aiming at. We can switch to full tiling mode to see what it all looks like. This looks like what we wanted.

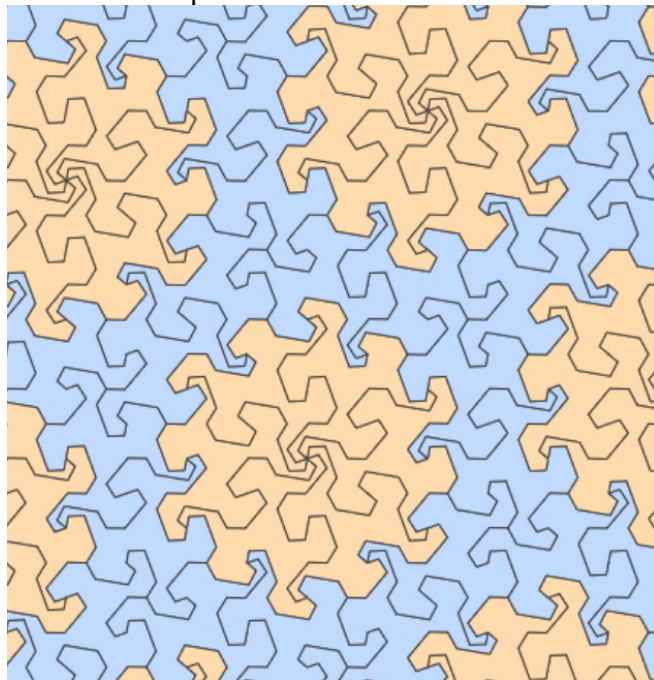
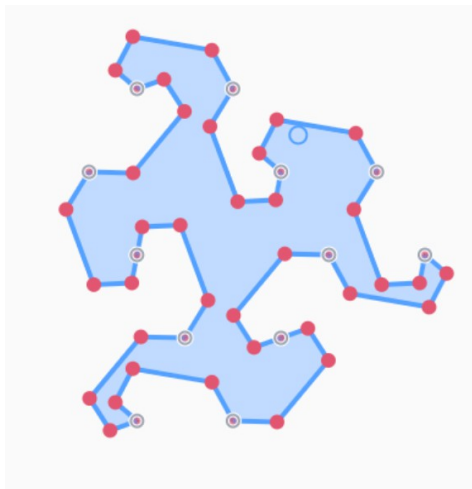
We can go a step further. Note that up above in green that it says “closes – reducible.” That means that we can find this tiling with fewer edges.



Hit the “reduce” button. We get this figure below.

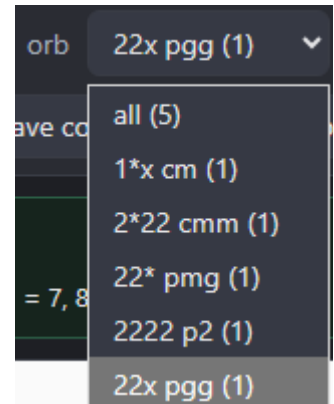


Notice this is the same shape, but now some of the vertices are colored red. These are curve points. Try dragging one a bit and you can change the curve. If you click and drag on an edge away from a control point, you will add another control point. Here's a random shape to illustrate it.

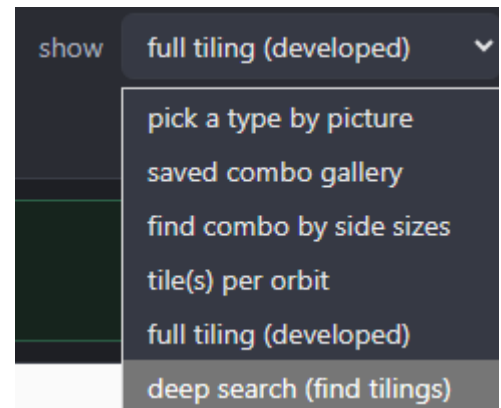


To make one on your own by computer:

First make sure that you have some type that is possible loaded. Refresh the page and it defaults to #1344, which is possible, but boring. In the orbifold selection, choose 22x. This loads Tegula type #600495.

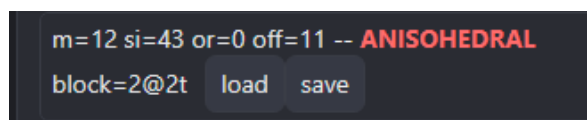


Select “deep search” mode.

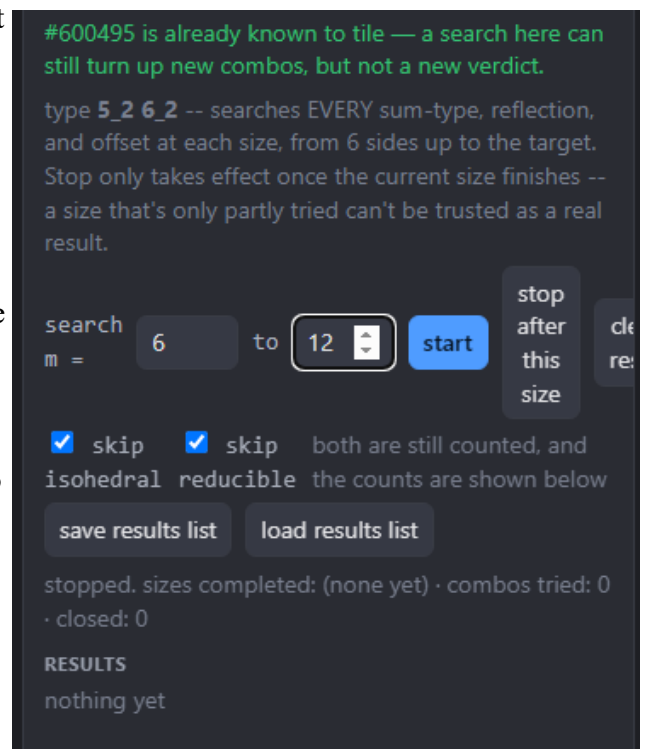
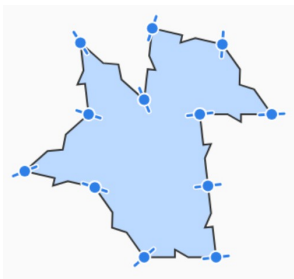


In the panel enter the start and stop m values for the m-gons. I checked the boxes to skip tiles that can tile isohedrally or that can be reduced. The computer does its best, but don't trust it too much. Most of the ones it gives are correct, but it may make mistakes. Usually, you can tell if the tile doesn't look right. It does Newton searches to find solutions and it may miss some.

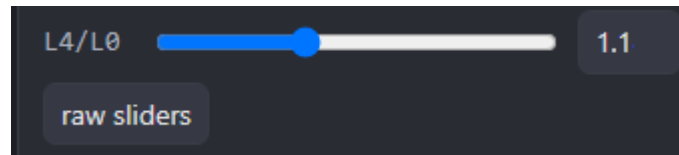
This is a long search. When it finishes, it has many anisohedral tilings it found. Choose the second from the



top, which is as shown just above. Then click on “demo curves”. You’ll see the figure below.



Notice the slider in the panel on the right. Try sliding this slowly and see how the shape changes. You can also put it into “full tiling” mode and see how the whole tiling changes as the length ratios changes. You can flip back to “tiles per orbit” and move the red dots to change the curve shape. Note how you have to hover over some curves to make the dots turn red. (Also note that some dots are green – if they are on a U curve.)



If you get a tiling you like, save it as an SVG or PNG file.

To make one on your own manually:

Each combo is separate and is made up of:

- Type: (number from Tegula program)
- m sides.
- Sum Index. (Which sides do we split into extra edges.)
- Orientation: 0 = direct. 1 = reflected
- offset: 0 to m-1. (These last two are how the two orbits map to each other. We can flip them or not and then rotate all the way around.)

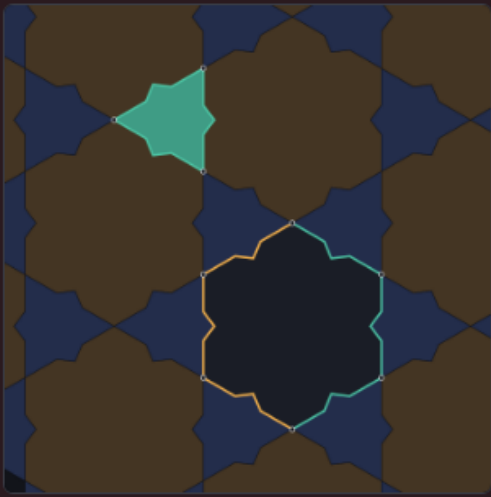
We can also pick the type by using $n_q\ k_r$ where the orbit tiles have n and k sides with q and r neighbors of the opposite orbit. Here's a list of all possible $n_q\ k_r$:

3_1 3_1, 3_1 3_2, 3_1 3_3, 3_2 3_2, 3_3 3_3, 3_1 4_1, 3_1 4_2, 3_1 4_4, 3_2 4_1, 3_2 4_2, 3_2 4_4,
 3_3 4_3, 3_1 5_1, 3_1 5_2, 3_2 5_1, 3_2 5_2, 3_2 5_4, 3_3 5_1, 3_3 5_3, 3_1 6_1, 3_1 6_2, 3_1 6_3,
 3_1 6_4, 3_1 6_6, 3_2 6_2, 3_2 6_4, 3_2 6_6, 3_3 6_1, 3_3 6_2, 3_3 6_3, 3_3 6_6, 3_1 7_1, 3_1 7_2,
 3_2 7_2, 3_2 7_4, 3_3 7_1, 3_3 7_3, 3_1 8_2, 3_1 8_4, 3_1 8_8, 3_2 8_2, 3_2 8_4, 3_2 8_8, 3_3 8_2,
 3_3 8_3, 3_3 8_6, 3_1 9_3, 3_1 9_6, 3_2 9_2, 3_2 9_4, 3_3 9_3, 3_1 10_4, 3_2 10_4, 3_2 10_8,
 3_3 10_6, 3_2 11_4, 3_1 12_4, 3_1 12_6, 3_1 12_8, 3_2 12_4, 3_2 12_6, 3_2 12_8, 3_3 12_6,
 3_1 12_12, 3_2 12_12, 3_1 16_8, 3_2 16_8, 3_2 18_8, 3_1 18_12, 3_1 24_12, 3_2 24_12, 4_1 4_1,
 4_1 4_2, 4_1 4_4, 4_2 4_2, 4_2 4_4, 4_3 4_3, 4_4 4_4, 4_1 5_1, 4_1 5_2, 4_2 5_1, 4_2 5_2,
 4_2 5_4, 4_3 5_3, 4_4 5_1, 4_4 5_2, 4_4 5_4, 4_1 6_2, 4_1 6_4, 4_1 6_6, 4_2 6_2, 4_2 6_3,
 4_2 6_4, 4_2 6_6, 4_3 6_3, 4_3 6_6, 4_4 6_1, 4_4 6_2, 4_4 6_4, 4_2 7_2, 4_2 7_4, 4_3 7_3,
 4_4 7_2, 4_4 7_4, 4_1 8_8, 4_2 8_2, 4_2 8_4, 4_2 8_8, 4_3 8_3, 4_3 8_6, 4_4 8_4, 4_2 9_4,
 4_2 9_6, 4_2 10_4, 4_2 10_8, 4_3 10_6, 4_2 12_6, 4_2 12_8, 4_1 12_12, 4_2 12_12, 4_2 18_12,
 5_1 5_1, 5_2 5_2, 5_2 5_4, 5_3 5_3, 5_4 5_4, 5_1 6_6, 5_2 6_2, 5_2 6_4, 5_2 6_6, 5_3 6_3, 5_3 6_6,
 5_4 6_4, 5_3 7_3, 5_4 7_4, 5_2 8_8, 5_3 8_6, 5_2 12_12, 6_1 6_6, 6_2 6_2, 6_2 6_4, 6_2 6_6,
 6_3 6_6, 6_4 6_4,

Here are some examples of how we can explore. Type in 6 in the “#” box. It says “proven impossible.” Click the “show proof picture” button. It shows a picture and brief explanation of why it’s impossible.

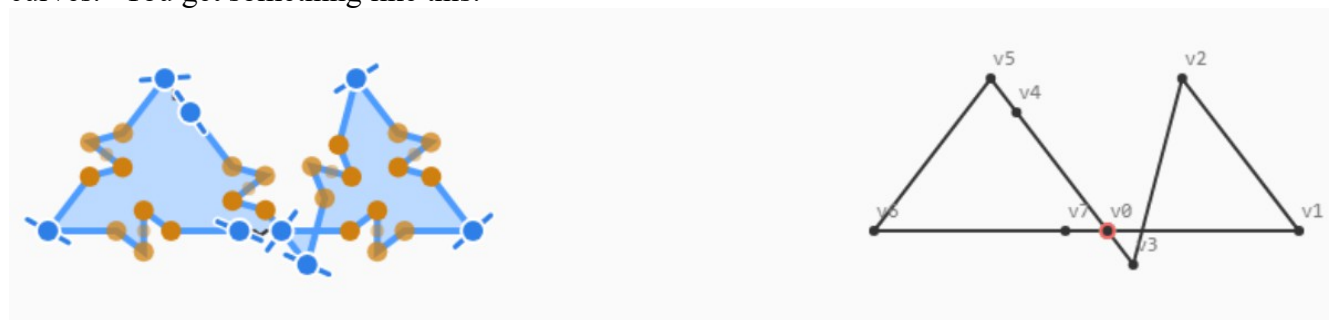
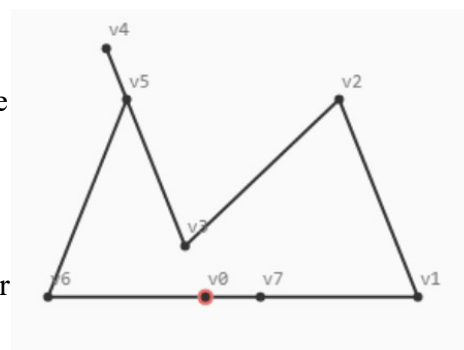
PROVEN IMPOSSIBLE #6 admits no tiling at all perimeter 6_6_3_3 · p6m · *632 · paper 3_3_6_6 - 1a MN p6m (#465) [hide proof picture](#)

No combination of sides, sum-type, reflection or offset can work — the proof is about the type itself. Anything drawn below is a shape the engine can build, not a counterexample. **Why:** The green tile's every edge borders the other orbit. On the other tile, edges of the SAME color match the edges of the green tile. But we have extra edges colored in a DIFFERENT color. This other tile has extra edges, so the perimeters can never be made equal.



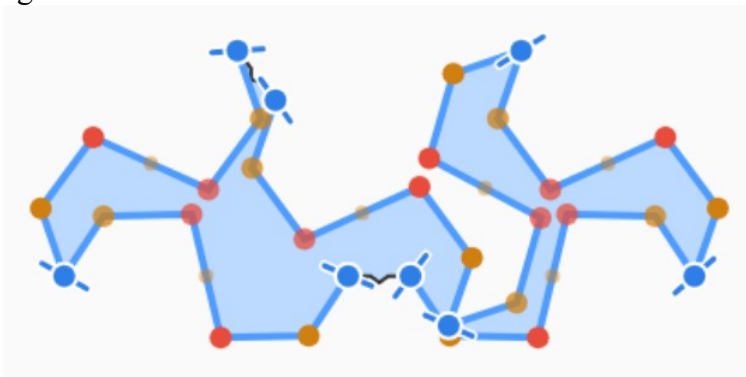
[open full size ↗ · netshape-6.png](#)

For another example, type in 25552 in the “#” box. Set the number of sides to 8. Set the sum index to #2. Check reflections on. Set the offset to 5. Then go to “tiles per orbit” mode. You get a weird shape that overlaps and collapses. Try sliding some of the sliders. You will see a dotted red line on the skeleton on the right as you move. That shows that the outline doesn’t close. The goal is to close this shape without overlaps... I have never been able to do this with straight lines. But, we can add curves to make it work. Set A1 slider to 52, A2 slider to 180, e0 slider to 1 and e4 slider to 0.22 Hit the “re-auto-solve” button to try to close the shape. Check the “demo curves.” You get something like this:

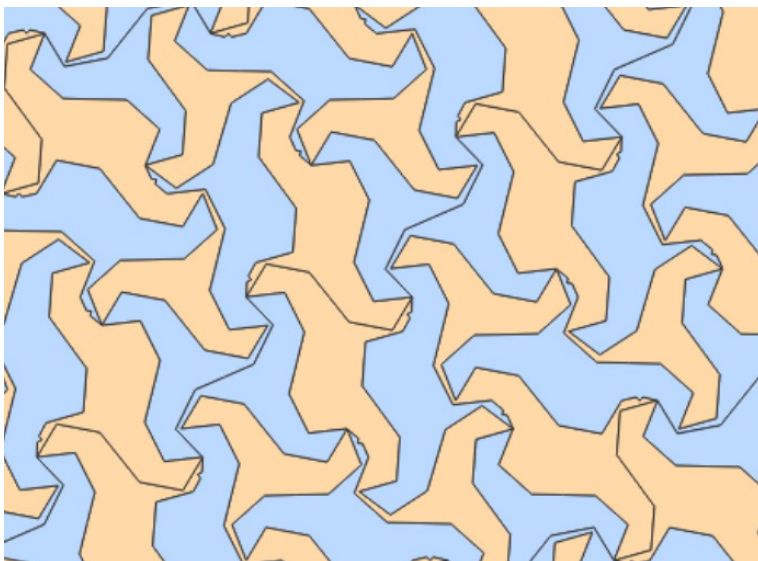


It closes, but has overlaps. Then drag the yellow dot on the lower right curve so it bends around vertex v3. Drag the dot next to it so it avoids the neighbor curve.

It should look something like this.



This tile now doesn't overlap and we can make a full tiling. (Granted this is a very tricky one. You can find a solution stored in the saved combo gallery under weird with label #25552b.) It's shown at the right.



Most combos can't make a tiling.

A big thank you to Olaf Delgado, Daniel Huson & Elizaveta Zamorzaeva for "The classification of 2-isohedral tilings of the plane." This article lists the 1270 types and started me exploring this area. I also stole the pictures from this article to illustrate the app. (Note to self - write them and ask for permission.)

This app started with the algorithm I gave at <https://johndberglund.github.io/anisohedral/algorithm.html> I worked with Claude AI to expand this to a workable program. It's still a work in progress.

Another big thank you to Daniel Huson, Rüdiger Zeller, and Klaus Westphal for program Tegula. I stole the Delauney-Dress symbols from their database. I also use their Tegula numbering system as a shortcut in my app. (Note to self - write them and ask for permission.) Here's a link:

<https://uni-tuebingen.de/en/fakultaeten/mathematisch-naturwissenschaftliche-fakultaet/fachbereiche/informatik/lehrstuehle/algorithms-in-bioinformatics/software/tegula/>

This let's you explore many more types of symmetry patterns beyond the 2-isohedral ones.